Dynamic Programming

Soontharee Koompairojn

Optimization Problem
(ปัญหาการหาค่าที่เหมาะสมที่สุด)

- *การเรียนรู้เพื่อหาคำตอบหรือวิธีการที่ดีที่สุดให้กับปัญหาจากคำตอบที่เป็นไปได้*
- *เป็นการหาค่าที่เหมาะสมที่สุด*
- *เป็นการหาค่าสูงสุดหรือค่าต่ำสุดของปัญหา*
- *แสดงปัญหาอยู่ในรูปของฟังก์ชันทางคณิตศาสตร์*

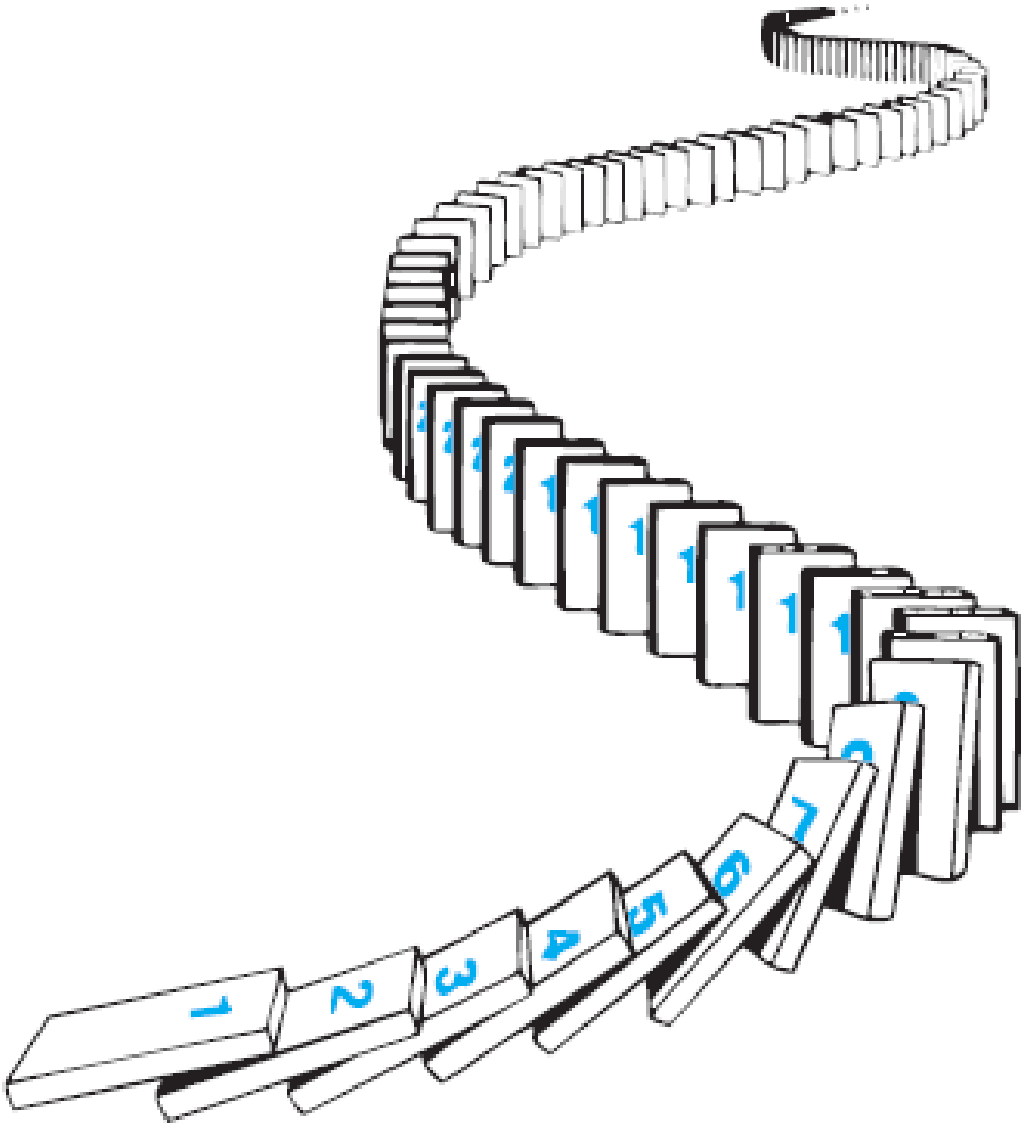
Algorithmic Paradigms

- Greedy. (Not optimal)
  - Build up a solution incrementally,
  - myopically optimizing some local criterion.

- Divide-and-conquer. (Optimal Solution)
  - Break up a problem into sub-problems,
  - solve each sub-problem independently, and
  - combine solution to sub-problems to form solution to original problem.

- Dynamic programming. (Optimal solution)
  - Break up a problem into a series of overlapping sub-problems,
  - And build up solutions to larger and larger sub-problems.

Ref: Algorithm design, Jon Kleinberg and Eva Tardos, slide by Kevin Wayne


1
2
3
4
5
6
7

Dynamic Programming Applications

- Areas.
  - Bioinformatics.
  - Control theory.
  - Information theory.
  - Operations research.
  - Computer science: theory, graphics, AI, compilers, systems, ….

- Some famous dynamic programming algorithms.
  - Unix diff for comparing two files.
  - Viterbi for hidden Markov models.
  - Smith-Waterman for genetic sequence alignment.
  - Bellman-Ford for shortest path routing in networks.
  - Cocke-Kasami-Younger for parsing context free grammars.

Ref: Algorithm design, Jon Kleinberg and Eva Tardos, slide by Kevin Wayne

Fibonacci Numbers

- ให้เขียน algorithm คำนวณ n^{th} Fibonacci number F_n
- $F_0 = 0$, $F_1 = 1$
- For all $i \geq 2$, $F_i = F_{i-1} + F_{i-2}$

- Recursive algorithm

Run time ?

Fibonacci Numbers

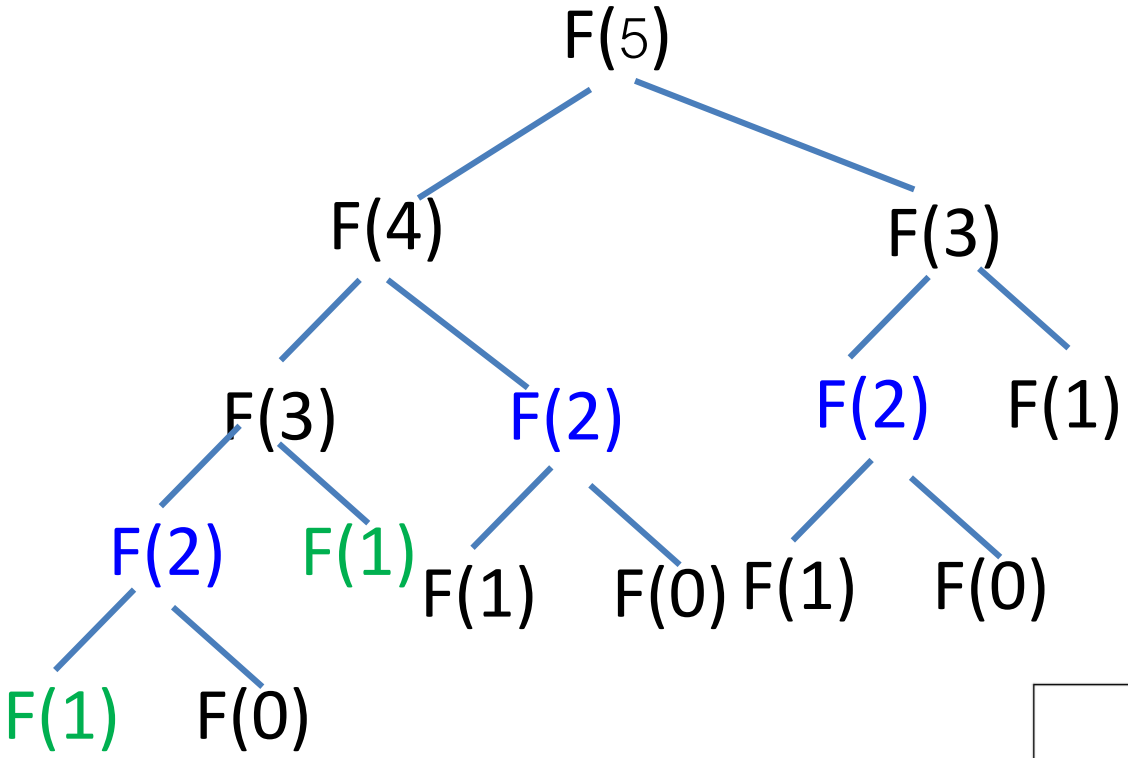
- Finding the nth Fibonacci number F_n, where,
- $F_0 = 0$, $F_1 = 1$
- For all $n \geq 2$, $F_n = F_{n-1} + F_{n-2}$

- <u>Recursive Algorithm</u>

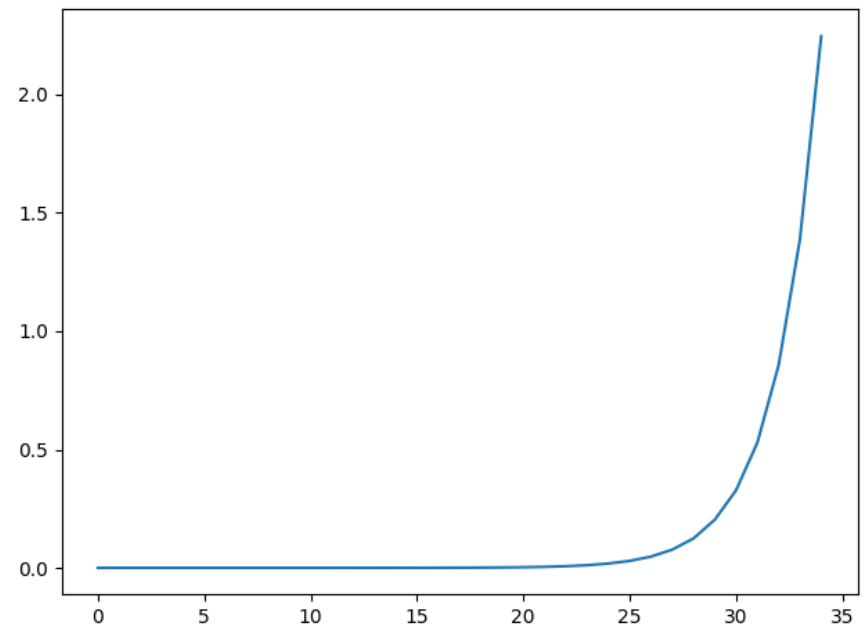
```
fibo(n){
 if (n<=1)
        f = n;
 else
      f = fibo(n-1)+fibo(n-2);
 Return  f;
}
```

- Run time

$$T(n) = O(((1+\sqrt{5})/2)^n$$



$$T(n) = T(n-1) + T(n-2) + O(1)$$



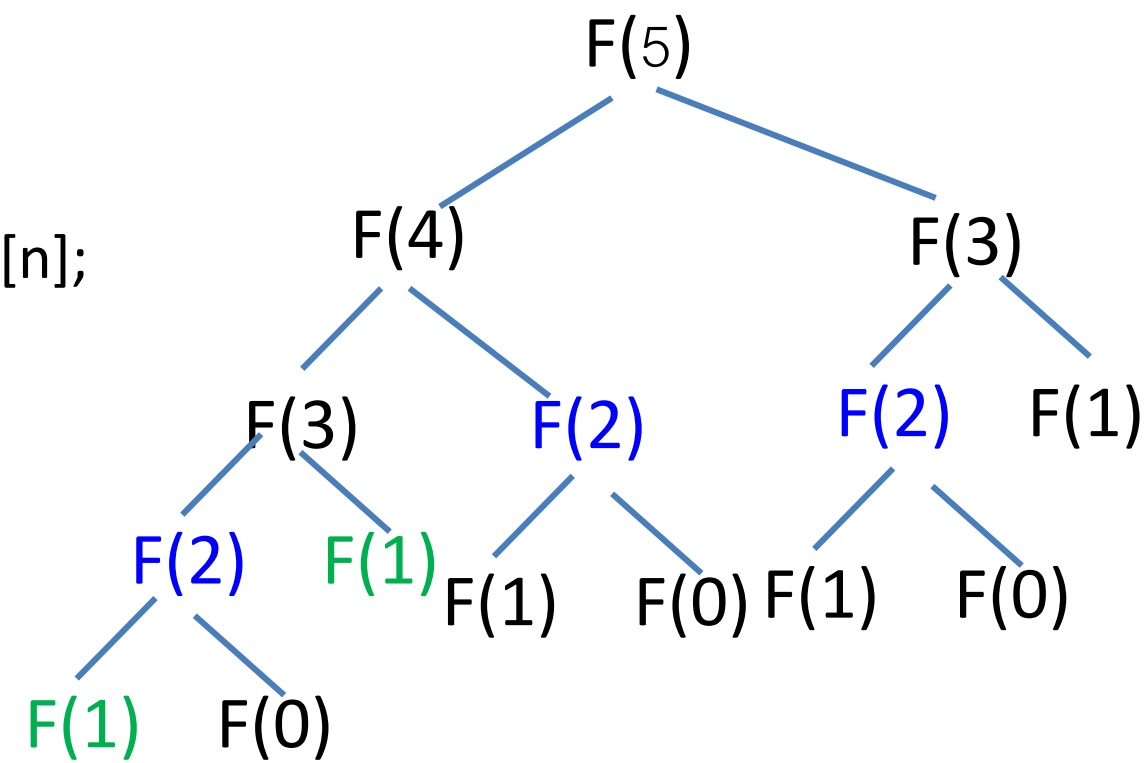
https://stackoverflow.com/

Finding the nth Fibonacci number F_n, where,
$F_0 = 0$, $F_1 = 1$
For all $i \geq 2$, $F_i = F_{i-1} + F_{i-2}$

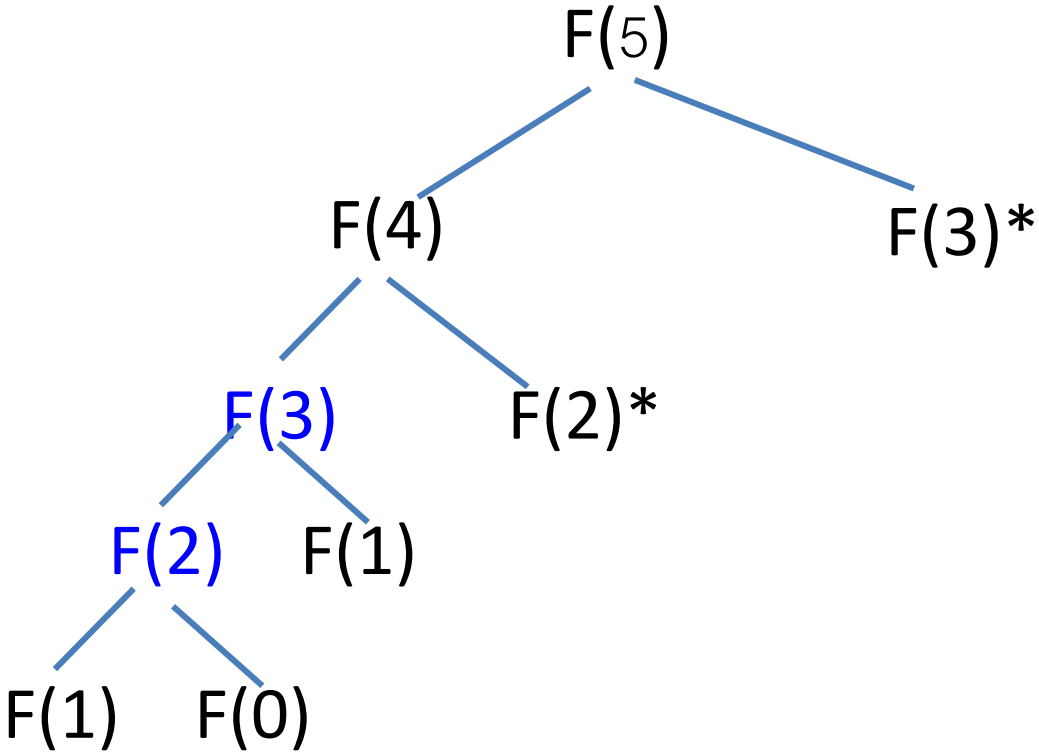
Top Down Dynamic Programming Algorithm

ใช้วิธีการจำคำตอบ (memorization) โดยใช้ array ➔ Tail recursive

```
fibo(n,M){
 if (n<=1)
        return n;
If (M[n]>0) return M[n];
F1 = fibo(n-1,M);
F2 = fibo(n-2,M);
M[n] = F1+F2;
Return  M[n];
}
```



Run time



$$T(n) = T(n-1) + O(2)$$
$$= O(n)$$

Finding the nth Fibonacci number F_n, where,
$F_0 = 0$, $F_1 = 1$
For all $i \geq 2$, $F_i = F_{i-1} + F_{i-2}$

Bottom-up Dynamic Programming Algorithm

```
int fibonacci(int n) {
   if (n == 1)
      return 0;
   else if (n == 2)
      return 1;
   else {
      int a = 1, b = 1, i, c;
      for (i = 3; i <= n; i++) {
         c = a + b;
         a = b;
         b = c;
      }
      return a;
   }
}
```

```
Fibo(n) {
Allocate M[0..n]
M[0]=0; M[1] = 1;
For (k = 2 to n) {
        M[k] = M[k-1]+M[k-2];
}
Return M[n]
}
```

Dynamic Programming

แนวทางการแก้ปัญหา

- กำหนดโครงสร้างการแก้ปัญหา optimal solution (ปัญหาที่ หาค่ามากที่สุด หรือ หาค่าที่น้อยที่สุด)
- คิดแก้ปัญหา optimal solution แบบ recursive
- คิดแก้ปัญหา optimal solution แบบ bottom-up
- คำนวณค่า optimal solution จากคำตอบที่ได้คำนวณมาก่อนหน้านี้

ปัญหา Maximum Subsequence Sum

กำหนดให้ S เป็นลำดับ (sequence) ตัวเลข n ตัว $<x_1, x_2, \ldots, x_n>$,

โดยอาจจะมีจำนวนลบอยู่ด้วย

ปัญหา คำนวณหา**ผลบวก**ของลำดับย่อย subsequencesของ S ที่มีค่ามากที่สุด

กำหนดให้ s' = subsequence ที่ต่อเนื่องของ S

$= x_i, x_{i+1}, \ldots, x_{i+k}$, โดยที่ $1 \le i \le n$ และ $k \ge 0$

สมมติ $S = \langle 1, -5, 2, -1, 3 \rangle$,

ตัวอย่างของ S' ได้แก่

$\langle 1 \rangle$, $\langle 2, -1, 3 \rangle$,และ $\langle -5, 2 \rangle$

MSS ของ sequence S เป็นเท่าไร?

Subsequence ≠ Subset

Maximum Subsequence Sum

{-2, -3, 4, -1, -2, 1, 5, -3}

Sum(1) = -2
Sum(2) = max{ Sum(1)+X[2], x[2])
= max{-2+-3, -3}
= -3
Sum(3) = max{sum(2)+X[3], X[3]}
= max[-3+4, 4}
= 4
Sum(4) = max{sum(3)+X[4], x[4]}
= max{4+(-1), -1}
= 3

Sum(5)=1
Sum(6)=2
Sum(7)=7
Sum(8)=4

Maximum subsequence sum problem(2)

Developing a dynamic programming algorithm

- กำหนดโครงสร้างของ optimal solution

  แบ่งปัญหาออกเป็นปัญหาย่อย

{-2, -3, 4, -1, -2, 1, 5, -3}

<u>แนวคิด</u> : เก็บผลรวม(Sum)ที่มากที่สุดของsubsequenceใดๆ สิ้นสุดที่ตัวเลข x_i, โดยที่ $1 \le i \le n$

กำหนดให้ B เป็น array 1 มิติขนาด n ตัว

B[i] เก็บค่า ผลบวกที่มากที่สุดของ subsequence ที่สิ้นสุดที่ตัวเลข x_i

เมื่อคำนวณค่าในตาราง B จะได้ค่าผลบวกที่มากที่สุดเอง

$S = \langle 1, -5, 2, -1, 3 \rangle$

B	1	2	3	4	5

$S = \langle 1, -5, 2, -1, 3 \rangle$

B	1	2	3	4	5

Pseudocode

$B[1] = x_1$

For $i = 1$ to $n-1$ do

$$B[i+1] = \begin{cases} B[i] + x_{i+1} & \text{if } B[i] > 0 \\ x_{i+1} & \text{if } B[i] \leq 0 \end{cases}$$

ปัญหา longest increasing subsequence(LIS)

กำหนดให้ S เป็นลำดับ (sequence) ตัวเลข n ตัว $<x_1, x_2, \ldots, x_n>$,

โดยอาจจะมีจำนวนลบอยู่ด้วย

ปัญหา หาลำดับย่อย subsequencesของ S $<x_{i1}, x_{i2}, \ldots, x_{ik}>$,

ที่เรียงลำดับจากน้อยไปมากที่ยาวที่สุด

โดยที่ $1 < i_1 < i_2 < \ldots < i_k$

และ $x_{i1} < x_{i2} < \ldots < x_{ik}$

สมมติ $S = \langle 2, 5, 1, 0, 7, 3, 4, 6, 99, 10, 8 \rangle$,

LIS Problem

- โจทย์ Find a Longest Increasing subsequence(LIS) of a sequence of a distinct integers
- Induction hypothesis (คิดครั้ง 1)

  Given a sequence of size < m,
  we know how to find an LIS of it.

  Base case: กรณีที่ sequence มีขนาด 1

  จะได้ว่า เรารู้คำตอบ

  Induction step:

  Given a sequence of size m,
  we find an LIS of its first m-1 elements,
  and consider x_m.

  ถ้า x_m มีค่ามากกว่า สมาชิกตัวสุดท้ายของ LIS แล้ว append x_m เข้าไปใน LIS

  มิฉะนั้น ยังไม่แน่ว่าจะทำอย่างไร เพราะอาจจะมี LIS อื่นที่เป็นคำตอบก็ได้

- Induction hypothesis (คิดครั้ง 2)

  Given a sequence of size < m,
  we know how to find ALL the LISs of it.
  Base case: กรณีที่ sequence มีขนาด 1

  จะได้ว่า เรารู้คำตอบ

  Induction step:

  Given a sequence of size m,
  we can check x_m against ALL of the LISs
  and find whether a longer IS exists.

Induction hypothesis (คิดครั้ง 3)

Given a sequence of size < m,
we know how to find the LIS of it
such that no other LIS of it has a smaller last number.

Base case: กรณีที่ sequence มีขนาด 1

จะได้ว่า เรารู้คำตอบ

Induction step:

Given x_m ,
we can determine whether it can be appended to LIS by the induction.
We need to know the best IS of length s-1,
then x_m may extend an IS of length s-2
making the new best IS of length s-1.

- Induction hypothesis (คิดครั้ง 4)

Given a sequence of size < m,
we know how to find the BIS(k)
for all k < m-1, it they exist.
[BIS(k) : The best increasing subsequence of length k,
the one that ends with the smallest number
BIS(k).last คือ the last number in the sequence BIS(k)]

Base case: กรณีที่ sequence มีขนาด 1

จะได้ว่า เรารู้คำตอบ

Induction step:

Given x_m ,
we can find which of the BISs it can change.

x_m extend IFF เงื่อนไข 2 อันนี้

1) x_m > BIS(k).last

2) x_m < BIS(k+1).last, so that BIS(k) with x_m ดีกว่า BIS(k+1)

นั่นคือ

BIS(1).last < BIS(2).last <…< BIS(s).last where s is the size of the LIS

Subset sum problem

กำหนดให้ เซต $S = \{a_1, a_2, \ldots, a_n\}$ เป็นเซตของเลขจำนวนเต็มบวก

และค่าผลรวมเป็นเลขจำนวนเต็ม B

คำถาม: มีเซตย่อย (subset) S' ของ S ที่ผลบวกของสมาชิกใน S'

มีค่าเท่ากับ B หรือไม่

ตัวอย่างเช่น กำหนดให้ เซต $S = \{3, 34, 4, 12, 5, 2\}$,

และค่าผลรวมเท่ากับ 11

คำถาม: มีเซตย่อย (subset) S' ของ S ที่ผลบวกของสมาชิกใน S'

มีค่าเท่ากับ 11 หรือไม่

มี หรือ ไม่มี , True or False ?

ตัวอย่าง

เซต S = <2,3>, n = 2

ผลรวม B = 5

Is there a subset (มี Subsetหรือไม่)?

True or False?

	ผลรวม j					
	0	1	2	3	4	5
a_1=2						
a_2=3						

ถ้า ค่าผลรวม j เป็น 0 นั่นคือ ไม่มีตัวเลขตัวไหนอยู่ในเซตคำตอบ

If j เป็น 0 then ให้เติม T ในช่องเพราะ**มีคำตอบ**ว่าเป็นเซตว่าง

ตัวอย่าง

เซต S = <2,3>, n = 2

ผลรวม B = 5

Is there a subset (มี Subsetหรือไม่)?

True or False?

	ผลรวม j					
	0	1	2	3	4	5
แถวที่ i → a_1=2	T					
a_2=3						

If a_i มีค่าผลรวมเท่ากับ j then ให้เติม T ในช่องตัดกัน

Else เติม F ในช่อง

ตัวอย่าง

เซต S = <2,3>, n = 2

ผลรวม B = 5

Is there a subset (มี Subsetหรือไม่)?

True or False?

						ผลรวม j
	0	1	2	3	4	5
แถวที่ i → a_1=2	T	F	T	F	F	F
a_2=3						

พิจารณา แถวแรก (i=1)

If a_i มีค่าผลรวมเท่ากับ j then ให้เติม T ในช่อง

Else เติม F ในช่อง

ตัวอย่าง

เซต S = <2,3>, n = 2

ผลรวม B = 5

Is there a subset (มี Subsetหรือไม่)?

True or False?

T(1,2) T(1,5)

	ผลรวม j					
	0	1	2	3	4	5
a_1=2	T	F	T	F	F	F
แถวที่ i → a_2=3	T		T	T		?

T(2,5)

สำหรับ แถวถัดไป(แถวที่ i=2) สำหรับแต่ละ **ผลรวม j**

If ผลรวม j ในแถวก่อนหน้านี้ เป็น T then ในแถวปัจจุบันนี้ก็มีค่า T ด้วย

(เรา**มีคำตอบ** แต่ไม่จำเป็นต้องใช้ a_i ในเซตคำตอบ)

If a_i มีค่าผลรวมเท่ากับ j then ให้เติม T ในช่อง

ตัวอย่าง

เซต S = <2,3>, n = 2

ผลรวม B = 5

Is there a subset (มี Subsetหรือไม่)?

True or False?

$T(1,2)=T(i-1,j-a_i)$

$T(1,5)=T(i-1,j)$

	ผลรวม j					
	0	1	2	3	4	5
$a_1=2$			T			F
แถวที่ i → $a_2=3$						

จาก $a_i + (j-a_i) = j$

$T(2,5)=T(i,j)$

ตัวอย่าง

เซต S = <2,3,5>, n = 3

ผลรวม B = 5

Is there a subset (มี Subsetหรือไม่)?

True or False?

	ผลรวม j					
	0	1	2	3	4	5
a_1=2						
a_2=3						
a_3=5						

if ตัวเลขตัวใดตัวหนึ่งในเซต S มีค่าเท่ากับ B ➔ T(n,B) เป็นจริง ---①

แนวคิดในการแก้ปัญหา

- ใช้แนวทางพิสูจน์ induction ช่วยแก้ปัญหา
- ให้ T(n,B) เป็นประพจน์ที่ใช้ในการพิสูจน์ว่า
  "มีเซตย่อย S' จำนวน n ตัวที่มีผลบวก= B หรือไม่"
- Base case:

  if ตัวเลขตัวใดตัวหนึ่งในเซต S มีค่าเท่ากับ B ➔ T(n,B) เป็นจริง ---①
- Induction step:
  - if T(n-1, B) เป็น**จริง,** ➔ ไม่ต้องใช้ a_n ---②
  - if T(n-1, B) เป็น**เท็จ,** ➔ อาจจะมี a_n

    ➔ ต้องดูว่า T(n-1, B-a_n) เป็นจริงหรือไม่---③

ดังนั้นปัญหา T(n,B) เราพิจารณาปัญหาย่อย T(n-1,B) และ T(n-1, B- a_n)

แนวคิดในการแก้ปัญหา

การจะแก้ปัญหา subset sum ที่มีผลรวม = B

เราต้องแก้ปัญหา subset sum ที่มี $\leq$ B ด้วย

กำหนดให้ เซต $S = \{a_1, a_2, \ldots, a_i \ldots, a_n\}$ และค่าผลรวม = B

ให้ เซต S' เป็นเซตตัวเลข i ตัว = $\{a_1, a_2, \ldots, a_i\}$

และค่าผลรวม = j โดยที่ $0 \leq j \leq B$

- ถ้า $j = 0$ นั่นคือ ไม่มีตัวเลขตัวไหนอยู่ในเซตคำตอบ
- ถ้า $j = a_i$ นั่นคือ a_i อยู่ในเซตคำตอบ ---①(Base case)
- ถ้า เซตตัวเลข i-1 ตัว = $\{a_1, a_2, \ldots, a_{i-1}\}$ มีค่าผลรวม = j
  ➔ เซตตัวเลข i ตัว = $\{a_1, a_2, \ldots, a_{i-1}, \mathbf{a_i}\}$ มีค่าผลรวม = j ด้วย ---②
- ถ้า เซตตัวเลข i-1 ตัว = $\{a_1, a_2, \ldots, a_{i-1}\}$ มีค่าผลรวม = $j - a_i$
  ➔ เซตตัวเลข i ตัว = $\{a_1, a_2, \ldots, a_{i-1}, \mathbf{a_i}\}$ จะมีค่าผลรวม = j ---③

ดังนั้นปัญหา $T(i,j)$ แบ่งออกเป็น
ปัญหาย่อย $T(i-1, j)$ และ $T(i-1, j- a_i)$

<u>แนวคิด</u> : เก็บคำตอบของปัญหาย่อยไว้ใน 2D Boolean array T,
ที่มีขนาด n แถว และ $B+1$ คอลัมน์.
ถ้าเซตย่อย (subset) $\{a_1, a_2, \ldots, a_i\}$ มีผลรวมเท่ากับ j
ให้ค่า $T[i,j] = \text{true}$
มิฉะนั้น ให้ค่า $T[i,j] = \text{false}$

Algorithm สำหรับ subset sum problem

สำหรับแถวแรก, (i=1)

```
for  0 ≤ j ≤ B
   if   (j=0) or (a1 = j)
            T[1,j]  = true
   else  T[1,j]  = false
```

สำหรับแถวที่ i, โดยที่ 2 ≤ i ≤ n

```
for  0 ≤ j ≤ B
   if ((T[i-1,j]==true) or (T[i-1,j- ai ] ==true))
                  T[i,j]    = true
   else           T[i,j]    = false
```

$T(1,2)=T(i-1,j-a_i)$ $T(1,5)=T(i-1,j)$

	ผลรวม j					
	0	1	2	3	4	5
$a_1=2$						
$a_2=3$						

$T(2,5)=T(i,j)$

ตัวอย่าง

เซต S = <2,3,5,6>

ผลรวม B = 16

Is there a subset (มี Subsetหรือไม่)?

True or False?

	ผลรวม B																
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a_1=2																	
a_2=3																	
a_3=5																	
a_4=6																	